

**MAS ALLA DEL FISICO: ANALISIS DE DATOS Y PREDDICION DE
MEDALLAS EN JUEGOS OLIMPICOS**

**LUIS MIGUEL CARRILLO GARCIA
LUIS FERNANDO JARAMILLO CAÑAS
DAVID ESTEBAN ORTIZ HERNANDEZ**

**TECNOLOGICO DE ANTIOQUIA INSTITUCIÓN UNIVERSITARIA
BASES DE DATOS AVANZADAS**

MEDELLÍN

2026



TABLA DE CONTENIDO

3. INTRODUCCIÓN.....	5
4. JUSTIFICACIÓN	6
5. OBJETIVOS	7
5.1. OBJETIVO GENERAL.....	7
5.2. OBJETIVOS ESPECÍFICOS.....	7
6. CAPÍTULO 1: COMPRENSIÓN DEL NEGOCIO.	8
6.1. Análisis Descriptivo:.....	8
6.2. Análisis de Diagnóstico:	8
6.3. Análisis Predictivo:.....	10
6.4. Análisis Prescriptivo:	10
7. CAPÍTULO 2: ENTENDIMIENTO DE LOS DATOS.	11
8. CAPÍTULO 3: PREPARACIÓN DE LOS DATOS.	15
9. CAPÍTULO 4: MODELADO.	18
10. CAPÍTULO 5: EVALUACIÓN.	25
11. CAPÍTULO 6: EVALUACIÓN Y DESPLIEGUE.....	37
12. CONCLUSIONES	60
13. REFERENCIAS.	61



1. ÍNDICE DE FIGURAS

Figura	Descripción	Página
Figura 1	Muestra del dataset original y dimensiones iniciales (271,116 filas)	4
Figura 2	Información del DataFrame tras eliminación de duplicados y limpieza de medallas	4
Figura 3	Cálculo de la media de altura (175.33 cm) para imputación de nulos	4
Figura 4	Resumen de nulos eliminados y exportación del archivo limpio	5
Figura 5	Cálculo de la media de edad (25.45 años) y verificación de tipos de datos	5
Figura 6	Conteo final de valores nulos por columna (ID, Name, Sex, etc.)	5
Figura 7	Confirmación de limpieza total del dataset antes del modelado	6



2. INDICE DE TABLAS

Tabla	Descripción	Página
Tabla 1	Diccionario de datos: Definición de campos, tipos de datos e importancia para el análisis de atletas olímpicos.	9



3. INTRODUCCIÓN

La presente investigación está enfocada en el análisis de una base de datos histórica de los juegos olímpicos, la cual recopila información detallada sobre los atletas que han participado en las juntas deportivas a lo largo de un siglo. Este conjunto de datos abarca variables desde la nación hasta fisiológicas como sexo, talla y peso, así como información sobre el deporte, el evento y las medallas obtenidas. A través de la aplicación de técnicas de análisis de datos y machine learning, este estudio busca explorar que características físicas y demográficas que han estado presente en los participantes de cada juego olímpicos



4. JUSTIFICACIÓN

El análisis de datos deportivos se ha convertido en una herramienta fundamental para la toma de decisiones estratégicas en el ámbito olímpico. El presente proyecto se justifica desde múltiples dimensiones:

Dimensión científica y analítica:

Los Juegos Olímpicos representan el conjunto de datos deportivos más completo de la historia moderna, con registros que abarcan desde 1896 hasta 2016, más de 270,000 participaciones y 230 países. Este volumen y diversidad de información constituye un escenario ideal para la aplicación de técnicas avanzadas de Business Intelligence y Machine Learning, permitiendo identificar patrones que serían invisibles al análisis convencional.

Dimensión tecnológica:

El proyecto justifica la necesidad de implementar dos ecosistemas tecnológicos complementarios — Microsoft (SSIS, SSAS, Power BI) y Python (Pandas, Plotly, SQLite) — para demostrar que la inteligencia de negocios moderna no depende de una única plataforma, sino de la capacidad de integrar herramientas según el contexto organizacional. Esta comparación permite evaluar criterios de facilidad de uso, flexibilidad, escalabilidad y costo, elementos críticos en la selección de tecnologías para proyectos empresariales reales.

Dimensión organizacional y estratégica:

Las organizaciones deportivas, comités olímpicos nacionales y entidades gubernamentales invierten millones en el desarrollo de atletas sin contar con modelos predictivos que orienten esa inversión. Este proyecto demuestra cómo los datos históricos pueden transformarse en inteligencia estratégica: identificar qué perfiles físicos tienen mayor probabilidad de éxito, qué países optimizan mejor sus recursos, y cómo ha evolucionado la participación femenina, son preguntas cuyas respuestas tienen impacto directo en políticas de captación de talento y planificación deportiva.

Dimensión académica:

En el marco del programa de Ingeniería del Tecnológico de Antioquia, este proyecto integra competencias de bases de datos avanzadas, análisis de datos, modelado dimensional y programación, aplicándolas a un caso real de alta complejidad. La metodología CRISP-DM empleada garantiza un proceso riguroso desde la comprensión del negocio hasta el despliegue, alineándose con los estándares de la industria de datos.



5. OBJETIVOS.

5.1. OBJETIVO GENERAL

- Implementar una solución de Inteligencia de Negocios mediante análisis descriptivos, diagnósticos y predictivos sobre los registros históricos de los Juegos Olímpicos para determinar la probabilidad de éxito de un atleta y optimizar la planeación de delegaciones nacionales

5.2. OBJETIVOS ESPECÍFICOS.

- Realizar un análisis descriptivo para caracterizar el perfil fisiológico (edad, talla, peso) de los competidores a través de la historia.
- Ejecutar un análisis diagnóstico que identifique la relación entre la inversión/desarrollo de un país y su eficiencia en la obtención de medallas.
- Desarrollar un modelo predictivo de machine learning que estime la probabilidad de ganar una medalla basándose en la persistencia del atleta y sus variables demográficas.



6. CAPÍTULO 1: COMPRENSIÓN DEL NEGOCIO.

Estos datos se basan en las tendencias del histórico de los juegos olímpicos, lo que hace adecuado para analizar y estudiar las principales características físicas y demográficas de cada participante de los juegos olímpicos y las sedes y como varían estos:

6.1. Análisis Descriptivo:

- ¿Cuál es el perfil demográfico y físico de los atletas que han participado en los Juegos Olímpicos a lo largo de la historia?
- ¿Cuál es el promedio de edad, altura y peso de los competidores olímpicos?
- ¿Qué porcentaje histórico de los atletas participantes logra ganar una medalla?
- ¿Cuántos datos incompletos (nulos) existen en los registros históricos y cómo afecta esto al análisis?
- ¿Cuál es el top de países con mayores participaciones?
- ¿Cuál es la distribución de participaciones por sexo (M/F)?
- ¿Cuál es el promedio de edad de los atletas por deporte?
- ¿Cómo ha evolucionado la participación femenina por año?
- ¿Cuáles son los 10 países con más medallas históricamente?
- ¿Cuál es el total entregado de medallas de Oro, Plata y Bronce?
- ¿En qué deportes se han entregado más medallas?
- ¿Qué ciudades han sido sede más veces de los Juegos Olímpicos?
- ¿Cómo ha evolucionado las participaciones a lo largo de la historia?

6.2. Análisis de Diagnóstico:

- ¿Existe una relación directa entre las características físicas de un atleta (edad, altura, peso) y su éxito para obtener una medalla olímpica?
- ¿Por qué las características biométricas por sí solas presentan una correlación tan baja (¿menor al 0?) con la obtención de medallas?
- ¿Qué nos dice esta falta de correlación sobre la importancia de otras variables (como el tipo de deporte o el país de origen) que no se están midiendo en este bloque?



- ¿Se debe la dominancia de ciertos países a su nivel de desarrollo económico o a su volumen poblacional?
- ¿De qué manera influye la sede de los juegos en el éxito de la nación anfitriona?



6.3. Análisis Predictivo:

- ¿Es posible predecir mediante un modelo de Machine Learning si un atleta ganará una medalla basándonos exclusivamente en su sexo, edad, altura y peso?
- ¿Qué nivel de precisión general (Accuracy) puede alcanzar un algoritmo de Bosques Aleatorios (Random Forest) con estos datos?
- ¿Por qué el modelo tiene dificultades para identificar a los ganadores (desbalance de clases) a pesar de tener una precisión global alta?
- ¿Qué probabilidad tiene un atleta de ganar una medalla en su segunda participación si mantiene su peso y altura constantes?
- ¿Cómo evolucionará la brecha de participación entre hombres y mujeres en las próximas ediciones?

6.4. Análisis Prescriptivo:

- ¿Qué estrategias y decisiones deben tomar los comités olímpicos y entrenadores para maximizar sus posibilidades de ganar medallas en el futuro, basándose en los hallazgos de los datos?
- ¿Cómo se debe reestructurar la captación de talento para no depender únicamente de perfiles físicos generales?
- ¿Por qué es necesario crear modelos predictivos divididos por cada disciplina deportiva en lugar de usar un modelo global?



7. CAPÍTULO 2: ENTENDIMIENTO DE LOS DATOS.

Nombre (Campo)	Descripción de la información que contiene	Tipo de Dato	Importancia para el Análisis
ID	Número identificador único asignado a cada atleta.	Int	Baja
Name	Nombre y apellido del atleta.	Varchar (30)	Baja
Sex	Sexo biológico del atleta (M para masculino, F para femenino).	Varchar (8)	Alta
Age	Edad del atleta en el momento de competir en esos Juegos Olímpicos.	int	Alta
Height	Altura del atleta medida en centímetros.	Float	Alta



Weight	Peso del atleta medido en kilogramos.	Float	Alta
Team	Nombre del equipo bajo el cual compitió el atleta (suele ser el país).	Varchar (30)	Media
NOC	Código de 3 letras del Comité Olímpico Nacional (ej. USA, CHN, ARG).	Varchar (5)	Media
Games	Combinación del año y la temporada de los juegos (ej. "1992 Summer").	Varchar (20)	Baja



Year	Año exacto en el que se celebraron los Juegos Olímpicos.	Date	Media
Season	Temporada de los juegos: Verano (<i>Summer</i>) o Invierno (<i>Winter</i>).	Varchar (8)	Media
City	Ciudad anfitriona donde se llevaron a cabo los juegos.	Varchar (30)	Baja
Sport	Deporte general en el que compite el atleta (ej. Natación, Atletismo).	Varchar (15)	Alta



Event	Categoría o evento específico dentro del deporte (ej. Natación 100m libres).	Varchar (30)	Alta
Medal	Medalla ganada: <i>Gold</i> (Oro), <i>Silver</i> (Plata), <i>Bronze</i> (Bronce) o <i>No Medal</i> (No ganó).	Varchar (8)	Crítica



8. CAPÍTULO 3: PREPARACIÓN DE LOS DATOS.

Para buena elaboración del proyecto y llegar al objetivo se necesita una transformación de datos y limpieza de estos, por a continuación se hicieron los siguientes pasos:

Eliminar datos duplicados

Identificar las columnas donde hay datos nulos y plantear una solución

Las columnas Age, Weight, Height y Medal con tienen datos nulos, esto puede debido a que antiguamente no se tenían registros detallados con el peso, edad o la altura en los juegos olímpicos del siglo XIX y en Medal puede ser debido a que solo se les coloca a las personas que ganaron una medalla, como esta base recopila todos los participantes es normal que haya tantos sin una medalla

Una solución que se tomo fue para las columnas Age, Weight, Height se decidió sacar la Media de cada columna y remplazarla con los datos nulos; Y para la columna Medal se decidió remplazar los datos nulos con 'No obtuvo Medalla'. esto se cambios se hicieron en Python

Esto fue para la eliminación de datos duplicados

```
print(data1.shape)
... (271116, 15)

data1= data1.drop_duplicates()

print(data1.shape)
(269731, 15)
```

Remplazar los valores nulos en la columna Medal

```
data1['Medal'] = data1['Medal'].fillna('No Medal')
print("Valores nulos en la columna 'Medal' después del reemplazo:")
print(data1['Medal'].isnull().sum())

Valores nulos en la columna 'Medal' después del reemplazo:
0
```



Reemplazar los valores nulos en la columna Height con la Media que se saco

```
df=pd.DataFrame(data1)

print(df['Height'].mean())

175.33895323752947

data1['Height'] = data1['Height'].fillna(175)
print("Valores nulos en la columna 'Medal' después del reemplazo:")
print(data1['Height'].isnull().sum())

Valores nulos en la columna 'Medal' después del reemplazo:
0
```

Reemplazar los valores nulos en la columna Weight con la Media que se saco

```
print(df['Weight'].mean())

70.70177806382202

data1['Weight'] = data1['Weight'].fillna(70)
print("Valores nulos en la columna 'Weight' después del reemplazo:")
print(data1['Weight'].isnull().sum())

... Valores nulos en la columna 'Weight' después del reemplazo:
0
```

Reemplazar los valores nulos en la columna Age con la Media que se saco

```
print(df['Age'].mean())

25.454776204227084

data1['Age'] = data1['Age'].fillna(25)
print("Valores nulos en la columna 'Age' después del reemplazo:")
print(data1['Age'].isnull().sum())

Valores nulos en la columna 'Age' después del reemplazo:
0
```



```
print(data1.isnull().sum())
```

ID	0
Name	0
Sex	0
Age	9474
Height	60171
Weight	62875
Team	0
NOC	0
Games	0
Year	0
Season	0
City	0
Sport	0
Event	0
Medal	231333

dtype: int64

La Cantidad de datos Nulos que habían antes de la transformación

```
print(data1.isnull().sum())
```

ID	0
Name	0
Sex	0
Age	0
Height	0
Weight	0
Team	0
NOC	0
Games	0
Year	0
Season	0
City	0
Sport	0
Event	0
Medal	0

dtype: int64

Una vez sabemos que ya no tenemos datos nulos podemos empezar con el Modelado



9. CAPÍTULO 4: MODELADO.

Explique y organice el modelado analítico de datos y agregar gráfico representativo.

1. Se procede con creación del primer df el cual lo llamamos TEAMS, el cual contiene el equipo y el NO

```
[ ]
teams = (
    df[['Team', 'NOC']]
    .drop_duplicates()
    .reset_index(drop=True)
)

teams.insert(0, 'team_id', teams.index + 1)
```

```
[ ]
teams.head()
```

	team_id	Team	NOC
0	1	China	CHN
1	2	Denmark	DEN
2	3	Denmark/Sweden	DEN
3	4	Netherlands	NED
4	5	United States	USA

```
teams.to_csv('equipos_Games.csv', index=False)
```

2. Se procede con creación del segundo df el cual lo llamamos Atletas , el cual contiene el id de concursante, el nombre, sexo, edad, estatura y peso

[+ Código](#) [+ Texto](#)

```
athletes = (
    df[['ID', 'Year', 'Name', 'Sex', 'Age', 'Height', 'Weight']]
    .drop_duplicates() # ¡Solo usa ID para determinar unicidad!
    .reset_index(drop=True)
)

# Luego agregamos el athlete_id
athletes.insert(0, 'athlete_id', athletes.index + 1)
```

```
athletes.head()
```

	athlete_id	ID	Year	Name	Sex	Age	Height	Weight
0	1	1	1992	A Dijiang	M	24.0	180.0	80.0
1	2	2	2012	A Lamusi	M	23.0	170.0	60.0
2	3	3	1920	Gunnar Nielsen Aaby	M	24.0	175.0	70.0
3	4	4	1900	Edgar Lindenau Aabye	M	34.0	175.0	70.0



3. Se crea otra tabla que sea la edicion de los juegos

```
▶ Edition = (  
    df[['Games', 'Year', 'Season', 'City']]  
    .drop_duplicates()  
    .reset_index(drop=True)  
)  
  
Edition.insert(0, 'edition_id', Edition.index + 1)
```

```
Edition.head()
```

	edition_id	Games	Year	Season	City
0	1	1992 Summer	1992	Summer	Barcelona
1	2	2012 Summer	2012	Summer	London
2	3	1920 Summer	1920	Summer	Antwerpen
3	4	1900 Summer	1900	Summer	Paris
4	5	1988 Winter	1988	Winter	Calgary

```
Edition.to_csv('Edition_Games.csv', index=False)
```




4 se crea tabla de Competicion

```
Competition = (  
    df[['Sport', 'Event']]  
    .drop_duplicates()  
    .reset_index(drop=True)  
)  
  
Competition.insert(0, 'Competition_id', Competition.index + 1)
```

```
Competition.head()
```

	Competition_id	Sport	Event
0	1	Basketball	Basketball Men's Basketball
1	2	Judo	Judo Men's Extra-Lightweight
2	3	Football	Football Men's Football
3	4	Tug-Of-War	Tug-Of-War Men's Tug-Of-War
4	5	Speed Skating	Speed Skating Women's 500 metres

```
Competition.to_csv('Competition_Games.csv', index=False)
```

Hacemos el Merge sobre la tabla principal con las otras tablas que creamos anteriormente

```
df = df.merge(athletes[['ID', 'Year', 'athlete_id']], on=['ID', 'Year'], how='left')  
df = df.merge(teams[['Team', 'NOC', 'team_id']], on=['Team', 'NOC'], how='left')  
df = df.merge(Edition[['Games', 'City', 'edition_id']], on= ['Games', 'City'], how = 'left')  
df = df.merge(Competition[['Sport', 'Event', 'Competition_id']], on= ['Sport', 'Event'], how = 'left')
```

Eliminamos las Columnas que ya no serán necesarias

Se eliminan columnas extraídas del DF principal

```
df_athletes_events = df.drop(columns=['ID', 'Team', 'NOC', 'Name', 'Sex', 'Age', 'Height', 'Weight', 'Year', 'Season', 'Games', 'City', 'Sport', 'Event'])
```



Asi quedaría nuestra tabla JJOO

```
cols = ['id_participacion'] + [col for col in df_athletes_events.columns if col != 'id_participacion']
df_athletes_events = df_athletes_events[cols]
df_athletes_events.head()
```

	id_participacion	Medal	athlete_id	team_id	edition_id	Competition_id
0	1	No Medal	1	1	1	1
1	2	No Medal	2	1	2	2
2	3	No Medal	3	2	3	3
3	4	Gold	4	3	4	4
4	5	No Medal	5	4	5	5

E

Ejecutamos todo para que nos arrojen los archivos planos

```
DFATLETAS=df_athletes_events.to_csv('JJ.OO.csv',index=False)

Se exporta el DF principal y le damos un nuevo nombre

df.to_csv('JuegosOlimpicos_olimpics.csv', index=False)

num_duplicados = df_athletes_events.duplicated().sum()
print(f"Número de filas duplicadas: {num_duplicados}")

# Si quieres ver las filas duplicadas
duplicadas = df_athletes_events[df_athletes_events.duplicated()]
print("\nfilas duplicadas:")
print(duplicadas)

... Número de filas duplicadas: 0

Filas duplicadas:
Empty DataFrame
Columns: [id_participacion, Medal, athlete_id, team_id, edition_id, Competition_id]
Index: []

df.groupby("athlete_id").size().sort_values(ascending=False).head()
```

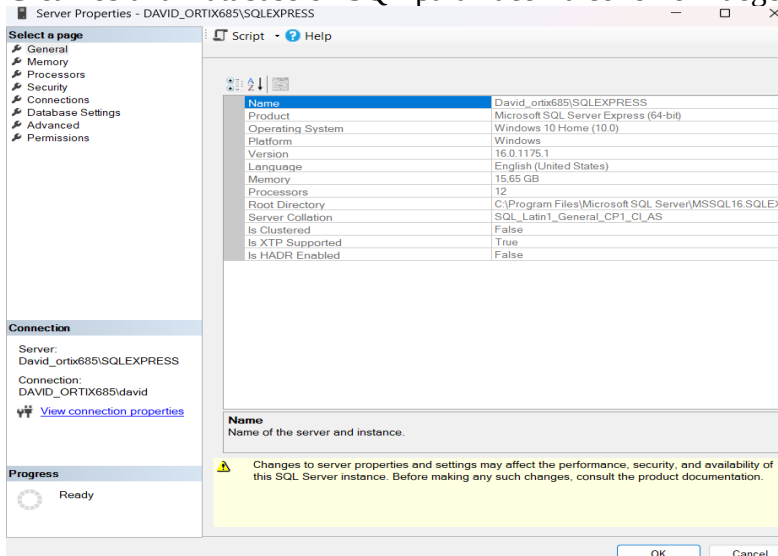


Sacamos todos los CSV que creamos para luego usarlos en el ETL

Windows File Explorer window showing the contents of the 'entregables_taller_bi' folder. The breadcrumb path is 'Inicio > David Esteban - Personal > Documentos > BIG DATA >'. The table below lists the files and folders in the directory.

Nombre	Estado	Fecha de modificación	Tipo	Tamaño
entregables_taller_bi		5/05/2026 2:27 p.m.	Carpeta de archivos	
Athletes_Games.csv		3/05/2026 10:09 p.m.	Archivo de valores...	10.295 KB
Competition_Games.csv		6/05/2026 12:35 a.m.	Archivo de valores...	39 KB
Edition_Games.csv		3/05/2026 10:08 p.m.	Archivo de valores...	2 KB
equipos_Games.csv		3/05/2026 10:08 p.m.	Archivo de valores...	25 KB
JJ.OO.csv		3/05/2026 10:08 p.m.	Archivo de valores...	8.185 KB

Creamos una Database en SQL para hacer la conexión luego al Visual





Creamos las tablas

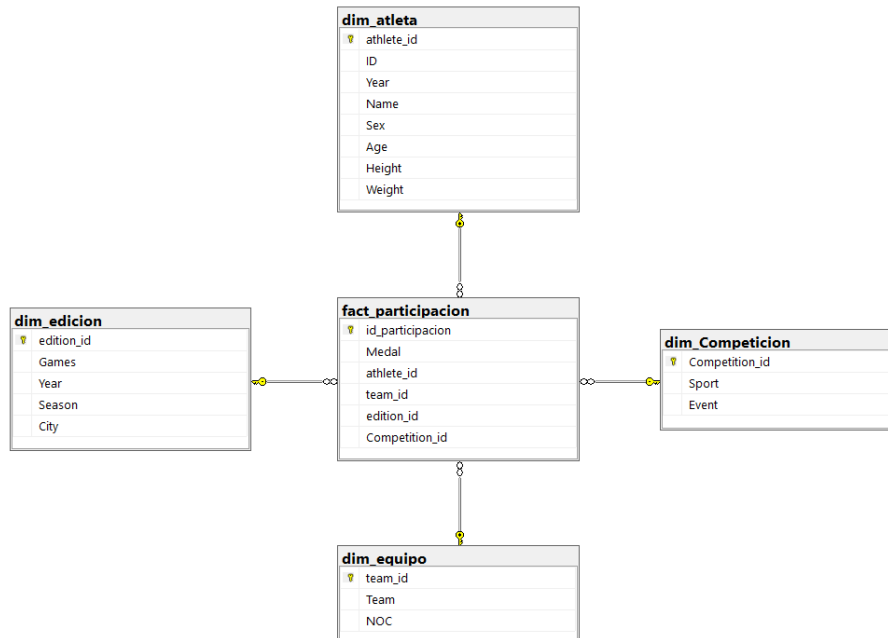
```
CREATE TABLE dim_edicion (  
    edition_id INT PRIMARY KEY,  
    Games VARCHAR(100),  
    Year INT,  
    Season VARCHAR(50),  
    City VARCHAR(100)  
);  
  
CREATE TABLE dim_atleta (  
    athlete_id INT PRIMARY KEY,  
    ID INT,  
    Year INT,  
    Name VARCHAR(200),  
    Sex CHAR(1),  
    Age VARCHAR(20),  
    Height VARCHAR(20),  
    Weight VARCHAR(20)  
);  
  
CREATE TABLE dim_equipo (  
    team_id INT PRIMARY KEY,  
    Team VARCHAR(100),  
    NOC VARCHAR(50)  
);  
  
CREATE TABLE dim_Competition (  
    Competition_id INT PRIMARY KEY,  
    Sport VARCHAR(150),  
    Event VARCHAR(300)  
);  
  
CREATE TABLE fact_participacion (  
    id_participacion INT PRIMARY KEY,
```

JuegosOlimpicos

- Database Diagrams
- Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.dim_atleta
 - dbo.dim_edicion
 - dbo.dim_equipo
 - dbo.dim_evento
 - dbo.fact_participacion
- Views
- External Resources
- Synonyms
- Programmability
- Query Store
- Service Broker
- Storage
- Security



Creamos el Diagrama de las tablas





10. CAPÍTULO 5: EVALUACIÓN.

Se entrega en la etapa final.

Ejecute y verifique la integración de los datos con el aplicativo SSIS de Microsoft, documentar cada paso.

Listar resultados.

Hacemos la conexión desde SQL a Visual

Connection Manager

Provider: Native OLE DB\Microsoft OLE DB Driver for SQL Server

OLE DB Provider: Microsoft OLE DB Driver for SQL Server

Enter a server or file name

Server or file name: David_ortix685\SQLEXPRESS

Location:

Log on to the server

☒ Use Windows NT Integrated Security

☐ Use a specific user name and password:

User name:

Password:

☐ Blank password

☐ Allow saving password

Initial catalog: JuegosOlimpico

Test Connection OK Cancel Help



Creamos los FLAT FILE para cada Archivo

FlatFile para JJOO

Flat File Connection Manager Editor

Connection manager name:

Description:

General
Columns
Advanced
Preview

Select a file and specify the file properties and the file format.

File name:

Locale: ☐ Unicode

Code page:

Format:

Text qualifier:

Header row delimiter:

Header rows to skip:

☒ Column names in the first data row



FlatFile para Equipos

Flat File Connection Manager Editor

Connection manager name:

Description:

General
Columns
Advanced
Preview

Select a file and specify the file properties and the file format.

File name:

Locale: ☐ Unicode

Code page:

Format:

Text qualifier:

Header row delimiter:

Header rows to skip:

☒ Column names in the first data row



FlatFile para Edition

Flat File Connection Manager Editor

Connection manager name:

Description:

General
Columns
Advanced
Preview

Select a file and specify the file properties and the file format.

File name:

Locale: ☐ Unicode

Code page:

Format:

Text qualifier:

Header row delimiter:

Header rows to skip:

☒ Column names in the first data row



FlatFile para Competition

Flat File Connection Manager Editor

Connection manager name:

Description:

General
Columns
Advanced
Preview

Select a file and specify the file properties and the file format.

File name:

Locale: ☐ Unicode

Code page:

Format:

Text qualifier:

Header row delimiter:

Header rows to skip:

☒ Column names in the first data row



FlatFile para Atletas

Flat File Connection Manager Editor

Connection manager name:

Description:

General
Columns
Advanced
Preview

Select a file and specify the file properties and the file format.

File name:

Locale: ☐ Unicode

Code page:

Format:

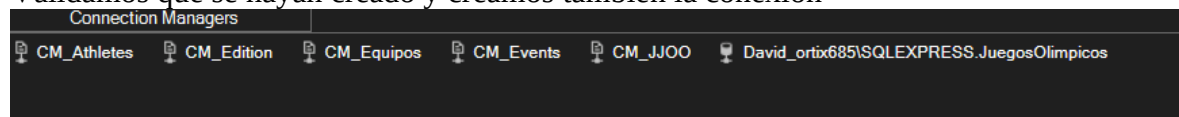
Text qualifier:

Header row delimiter:

Header rows to skip:

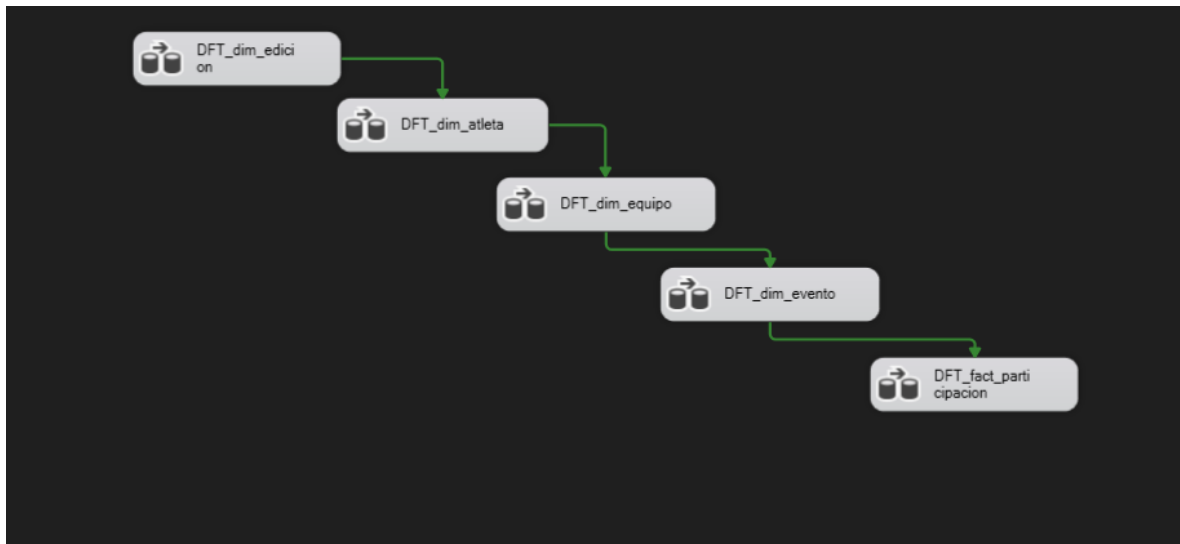
☒ Column names in the first data row

Validamos que se hayan creado y creamos también la conexión



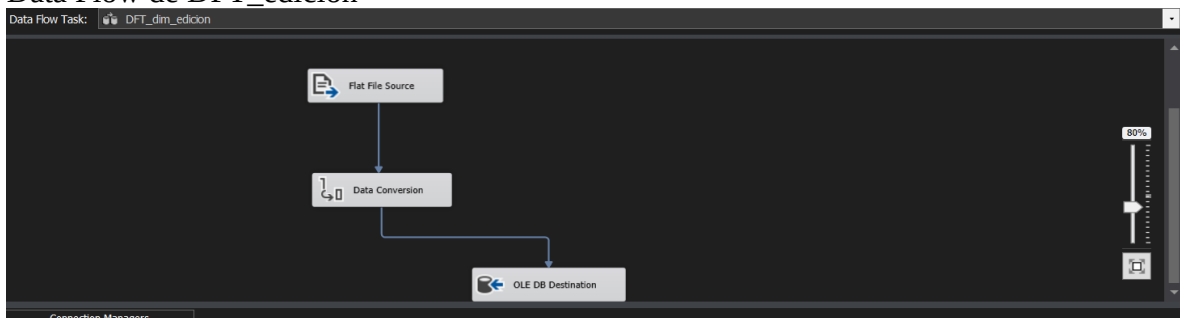


Hacemos el Data Flow para que todos los DFT se conecten

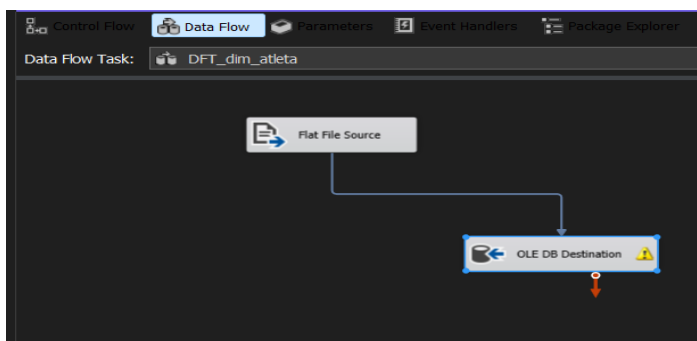


Hacemos el DATA Flow de cada DFT

Data Flow de DFT_edicion

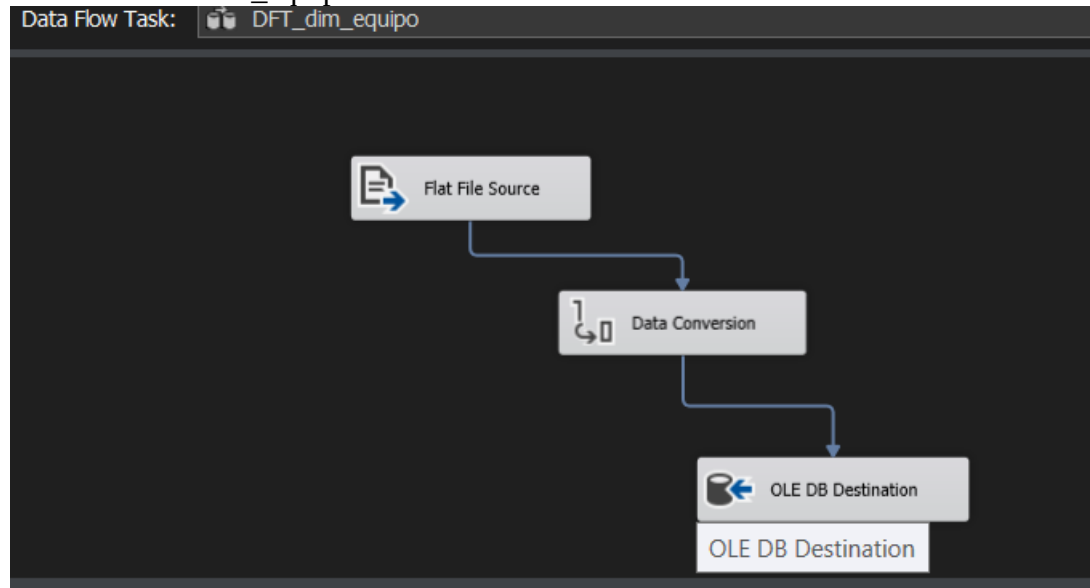


Data Flow de DFT_atleta

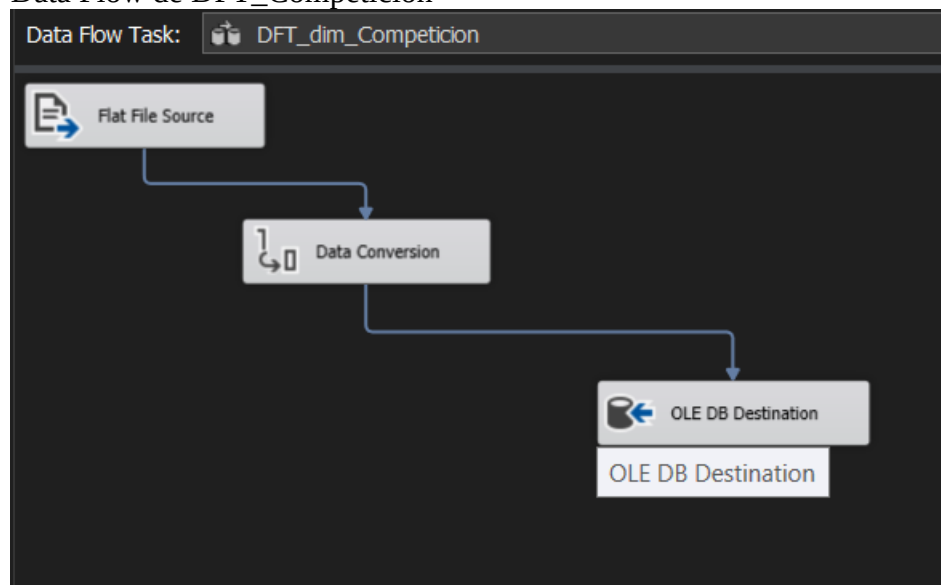




Data Flow de DFT_equipo

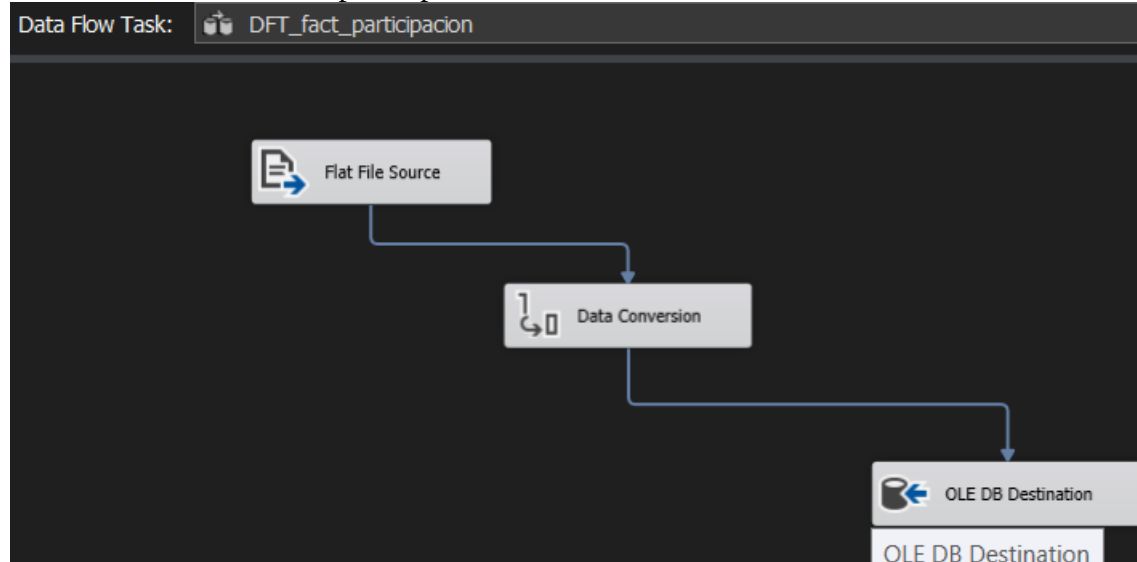


Data Flow de DFT_Competicion

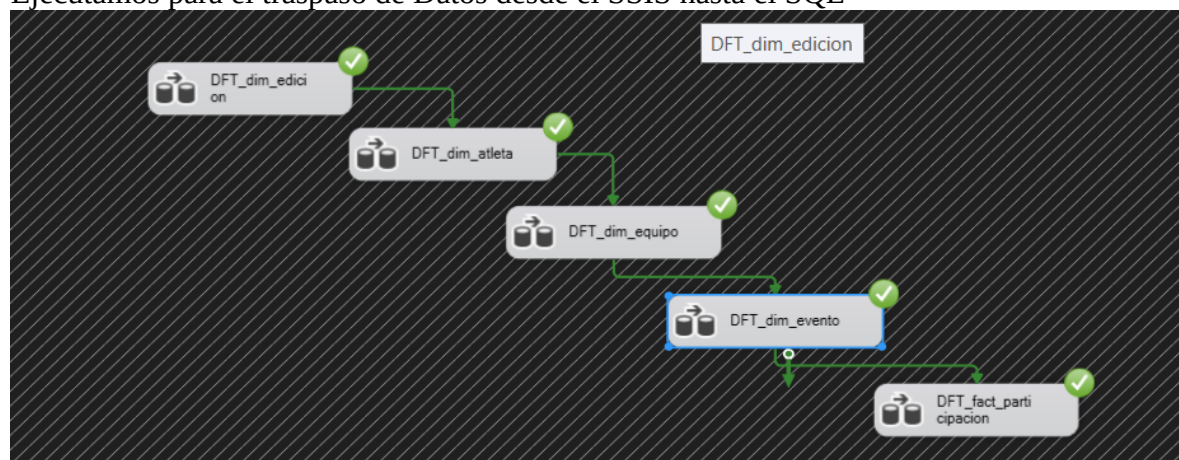




Data Flow de DFT_Fact_participacion



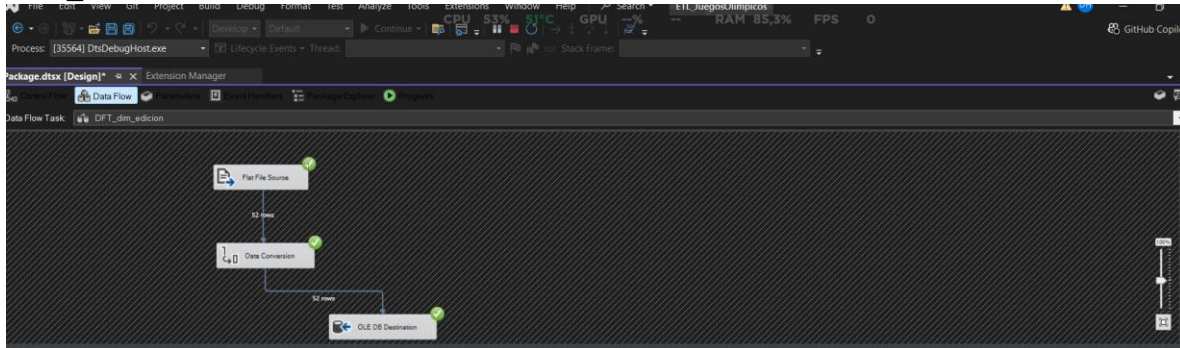
Ejecutamos para el traspaso de Datos desde el SSIS hasta el SQL



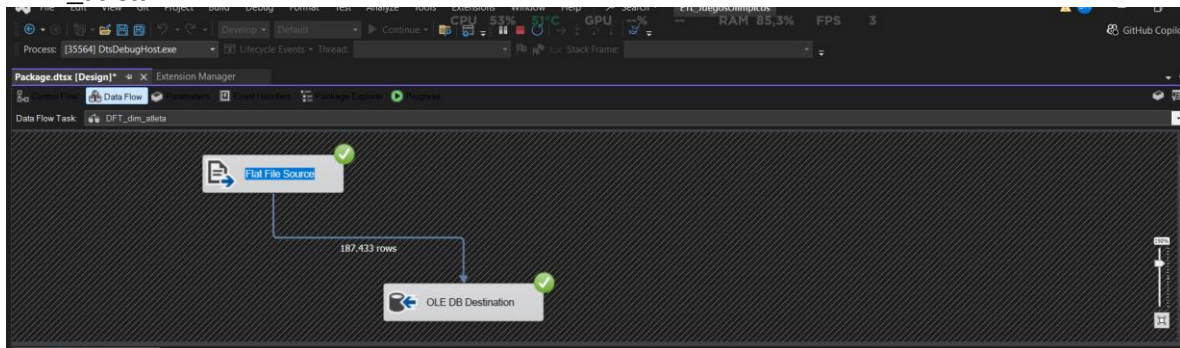


El numero de filas que se agregaron en cada DFT

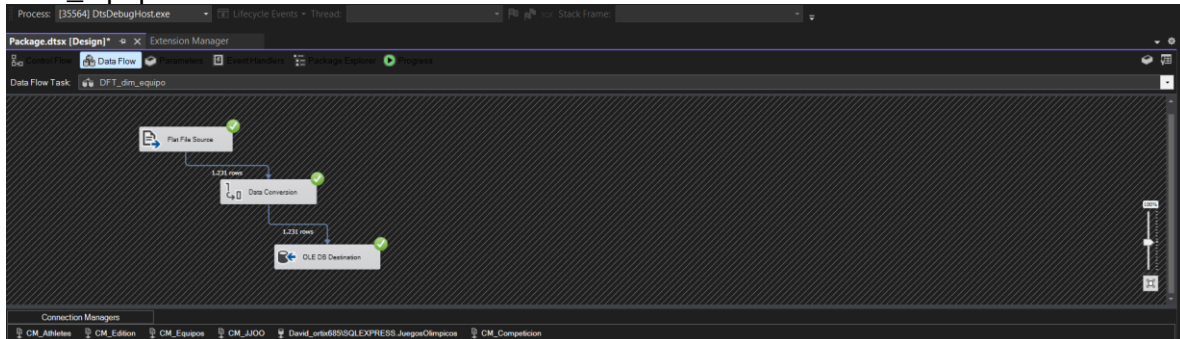
DFT_edicion



DFT_atleta

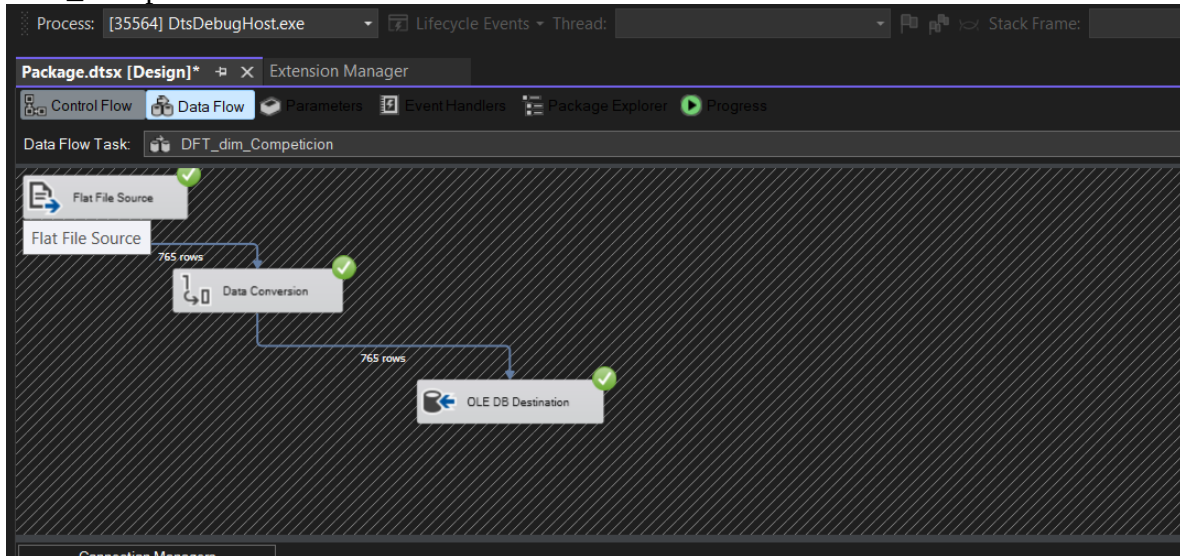


DFT_equipo

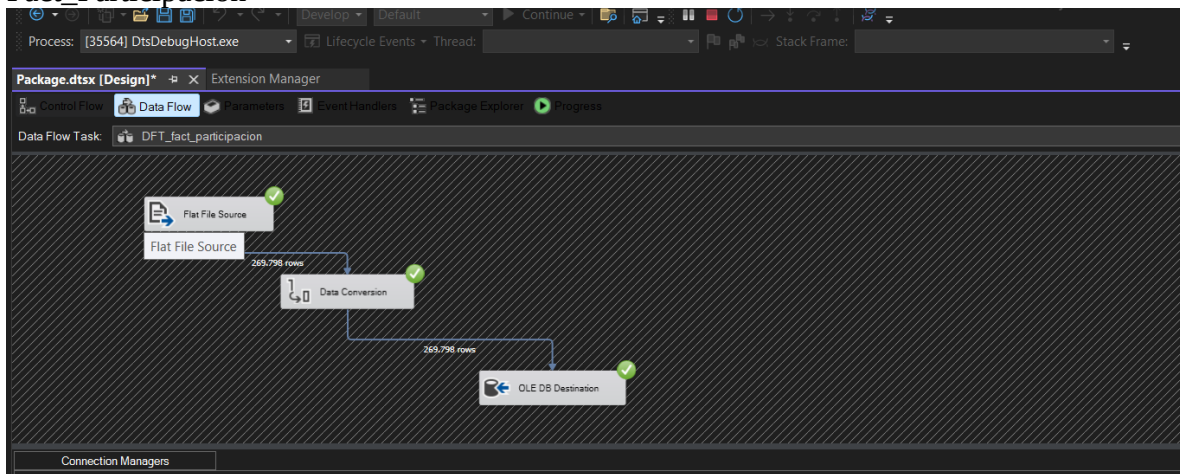




DFT_Competicion



Fact_Participacion





Luego en el SQL validamos que se hayan cargado los datos

```
USE JuegosOlimpico;  
SELECT 'dim_edicion' AS tabla, COUNT(*) AS filas FROM dim_edicion  
UNION ALL  
SELECT 'dim_atleta', COUNT(*) FROM dim_atleta  
UNION ALL  
SELECT 'dim_equipo', COUNT(*) FROM dim_equipo  
UNION ALL  
SELECT 'dim_Competicion', COUNT(*) FROM dim_Competicion  
UNION ALL  
SELECT 'fact_participacion', COUNT(*) FROM fact_participacion;
```

98 %

Results Messages

	tabla	filas
1	dim_edicion	52
2	dim_atleta	187433
3	dim_equipo	1231
4	dim_Competicion	765
5	fact_participacion	269798

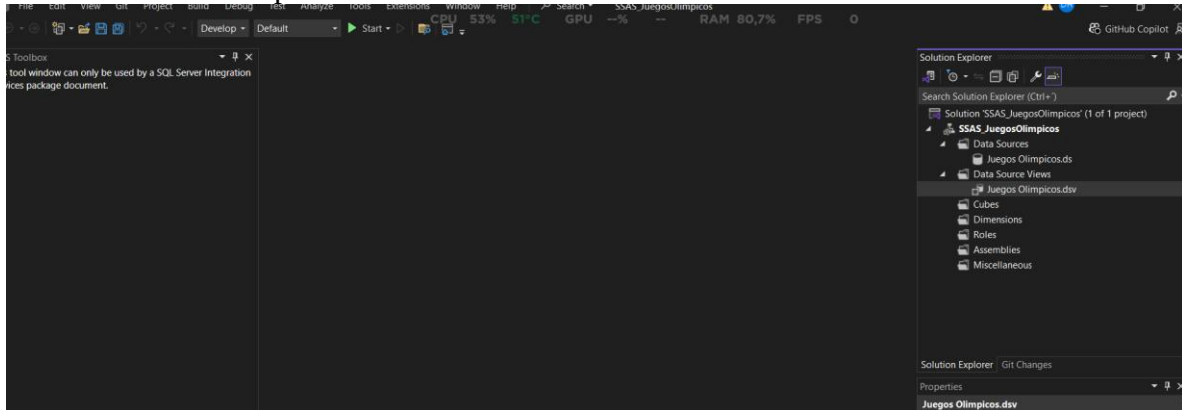


11. CAPÍTULO 6: EVALUACIÓN Y DESPLIEGUE

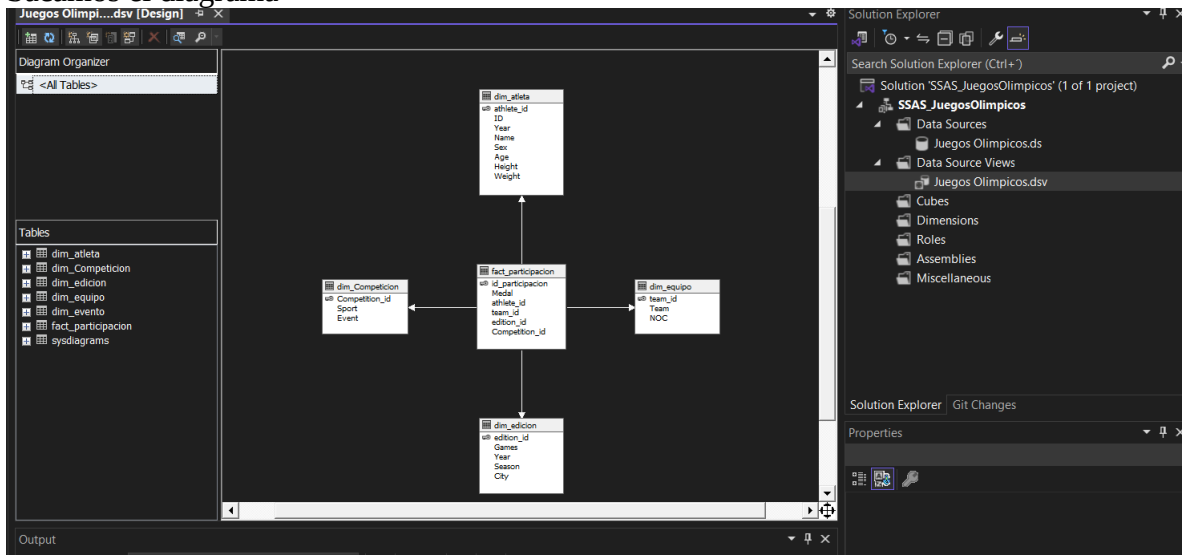
Se entrega en la etapa final.

Documente la realización del cubo y su integración con Power BI, generar un Dashboard y explicar los resultados obtenidos

Comenzamos con el proceso de la realización del Cubo

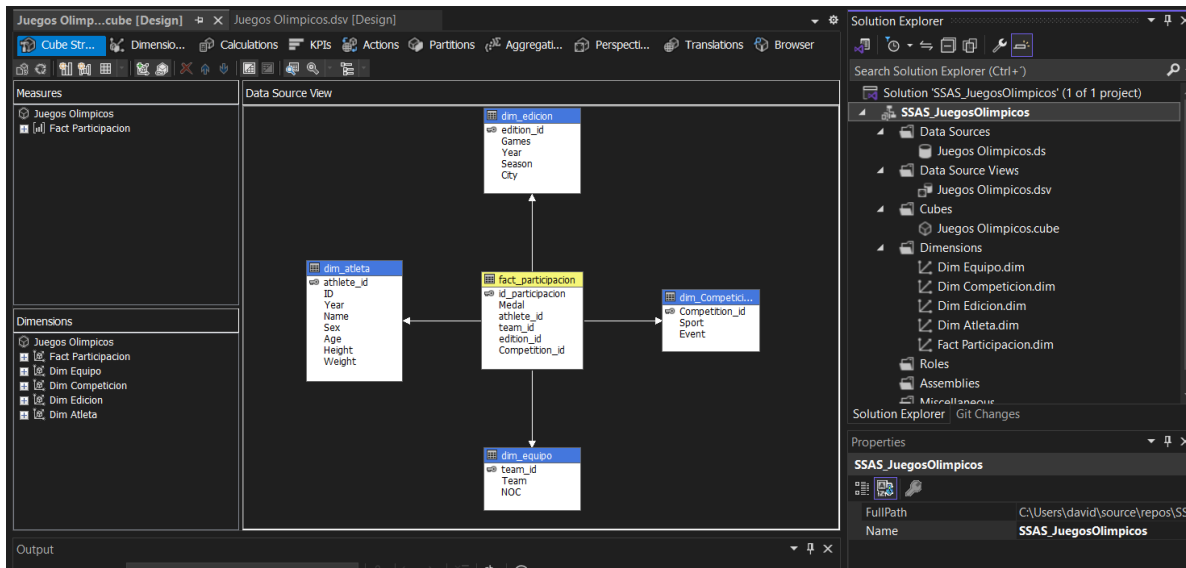


Sacamos el diagrama





Creamos el CUBO



Creamos el DataBase, seleccionando el Tipo de servicio: Analysis Services, con el nombre del servidor localhost



Añadimos los atributos necesarios en cada Dimension

Dim_atleta

Dimension Structure tool interface for Dim_atleta. The Attributes pane lists: Dim Atleta, Age, Athlete Id, Height, ID, Name, Sex, Weight. The Hierarchies pane is empty. The Data Source View pane shows the dim_atleta table with attributes: athlete_id, ID, Year, Name, Sex, Age, Height, Weight. The Output pane shows the deployment progress, including the build and deployment steps. The Deployment Progress pane shows the server as localhost and the database as SSAS_JuegosOlimpicos, with a green checkmark indicating successful deployment.

Dim_Competicion

Dimension Structure tool interface for Dim_Competicion. The Attributes pane lists: Dim Competicion, Competition Id, Event, Sport. The Hierarchies pane is empty. The Data Source View pane shows the dim_Competicion table with attributes: Competition_id, Sport, Event. The Output pane shows the deployment progress, including the build and deployment steps. The Deployment Progress pane shows the server as localhost and the database as SSAS_JuegosOlimpicos, with a green checkmark indicating successful deployment.



Dim_equipo

Attributes

- Dim Equipo
 - NOC
 - Team
 - Team Id

Hierarchies

To create a new hierarchy, drag an attribute here.

Data Source View

dim_equipo

- team_id
- Team
- NOC

Deployment Progress - SSAS_JuegosOlim...

Server: localhost
Database: SSAS_JuegosOlimpicos

Status:

Deployment Completed Successfully

Output

Show output from: Build

```
Add Dimension Dim Equipo
Add Cube Juegos Olimpicos
Done
Sending deployment script to the server...
Done
Deploy complete -- 0 errors, 0 warnings
===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====
===== Build completed at 11:45:00 AM and took 03.621 seconds =====
```

Dim_edicion

Attributes

- Dim Edicion
 - City
 - Edition Id
 - Games
 - Season
 - Year

Hierarchies

To create a new hierarchy, drag an attribute here.

Data Source View

dim_edicion

- edition_id
- Games
- Year
- Season
- City

Deployment Progress - SSAS_JuegosOlim...

Server: localhost
Database: SSAS_JuegosOlimpicos

Status:

Deployment Completed Successfully

Output

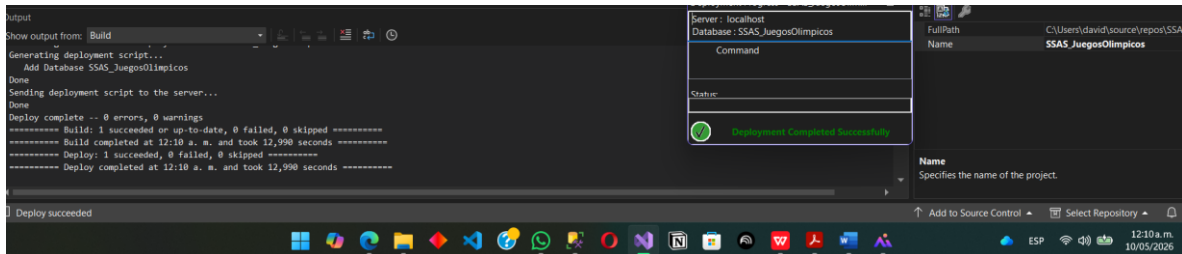
Show output from: Build

```
Add Dimension Dim Equipo
Add Cube Juegos Olimpicos
Done
Sending deployment script to the server...
Done
Deploy complete -- 0 errors, 0 warnings
===== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped =====
===== Build completed at 11:45:00 AM and took 03.621 seconds =====
```

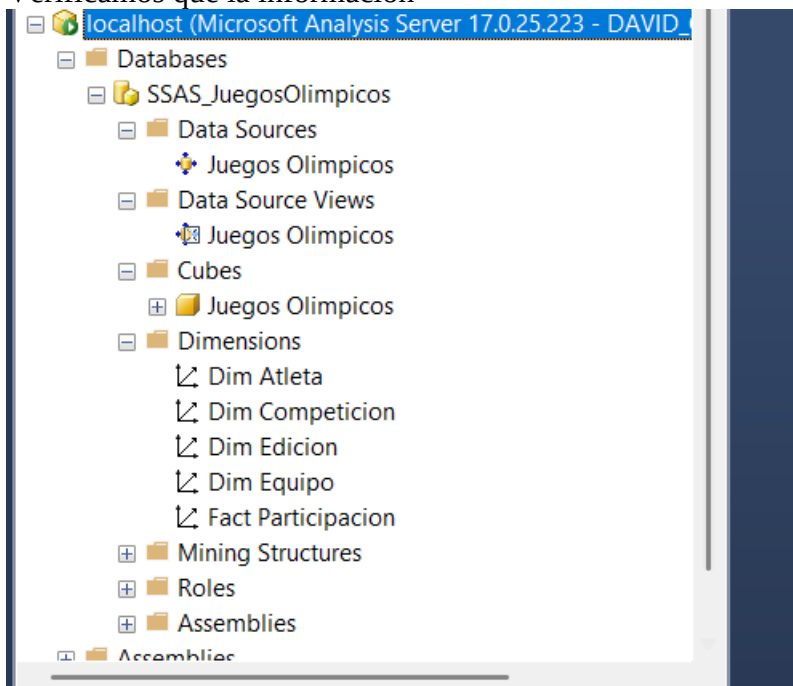


Dimensiones Creadas del Cubo analitico

Hacemos el Deploy



Verificamos que la información





Creamos las Medidas

	Command
1	CALCULATE
2	Porcentaje_Medallistas
3	Total_Participaciones
4	Total_Medallas
5	Tota_Oro
6	Total_Plata
7	Total_Bronce

Script Organizer

	Command
1	CALCULATE
2	Porcentaje_Medallistas
3	Total_Participaciones
4	Total_Medallas
5	Tota_Oro
6	Total_Plata
7	Total_Bronce

Calculation Tools

Name:

Porcentaje_Medallistas

Parent Properties

Parent hierarchy: Measures

Parent member:

Expression

$$\frac{([Measures].[Total Medallas]}{[Measures].[Fact Participacion Count]}) * 100$$

No issues found

Script Organizer

	Command
1	CALCULATE
2	Porcentaje_Medallistas
3	Total_Participaciones
4	Total_Medallas
5	Tota_Oro
6	Total_Plata
7	Total_Bronce

Calculation Tools

Name:

Total_Participaciones

Parent Properties

Parent hierarchy: Measures

Parent member:

Expression

$$[Measures].[Fact Participacion Count]$$

No issues found



Script Organizer

Command
1 CALCULATE
2 Porcentaje_Medallistas
3 Total_Participaciones
4 Total_Medallas
5 Tota_Oro
6 Total_Plata
7 Total_Bronce

Calculation Tools

Name: Total_Medallas

Parent Properties

Parent hierarchy: Measures

Parent member:

Expression

```
(([Measures].[Fact Participacion Count]) -  
([Measures].[Fact Participacion Count],  
No issues found
```

Script Organizer

Command
1 CALCULATE
2 Porcentaje_Medallistas
3 Total_Participaciones
4 Total_Medallas
5 Tota_Oro
6 Total_Plata
7 Total_Bronce

Calculation Tools

Name: Tota_Oro

Parent Properties

Parent hierarchy: Measures

Parent member:

Expression

```
(([Measures].[Fact Participacion Count],  
[Dim Fact Participacion].[Medal].&[Gold])  
No issues found
```

Script Organizer

Command
1 CALCULATE
2 Porcentaje_Medallistas
3 Total_Participaciones
4 Total_Medallas
5 Tota_Oro
6 Total_Plata
7 Total_Bronce

Calculation Tools

Name: Total_Plata

Parent Properties

Parent hierarchy: Measures

Parent member:

Expression

```
(([Measures].[Fact Participacion Count],  
[Dim Fact Participacion].[Medal].&[Silver])  
No issues found
```

Script Organizer

Command
1 CALCULATE
2 Porcentaje_Medallistas
3 Total_Participaciones
4 Total_Medallas
5 Tota_Oro
6 Total_Plata
7 Total_Bronce

Calculation Tools

Name: Total_Bronce

Parent Properties

Parent hierarchy: Measures

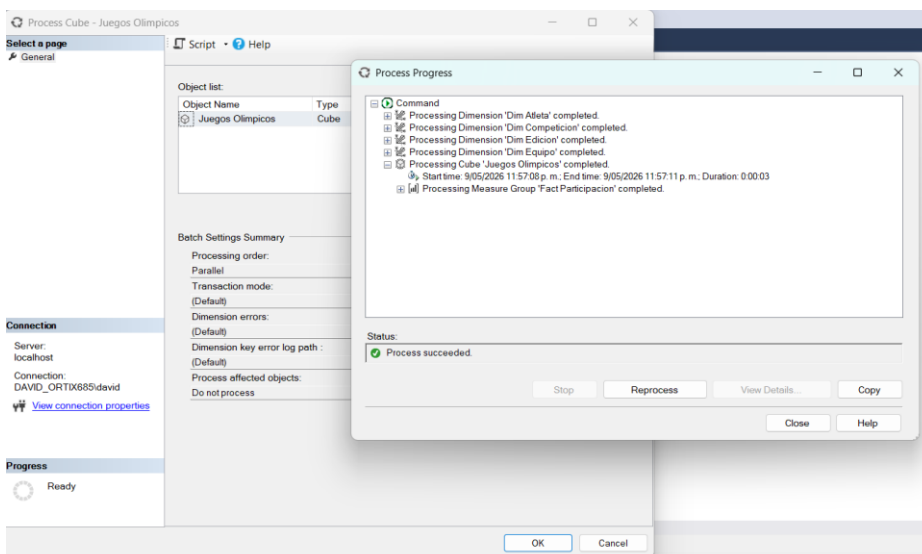
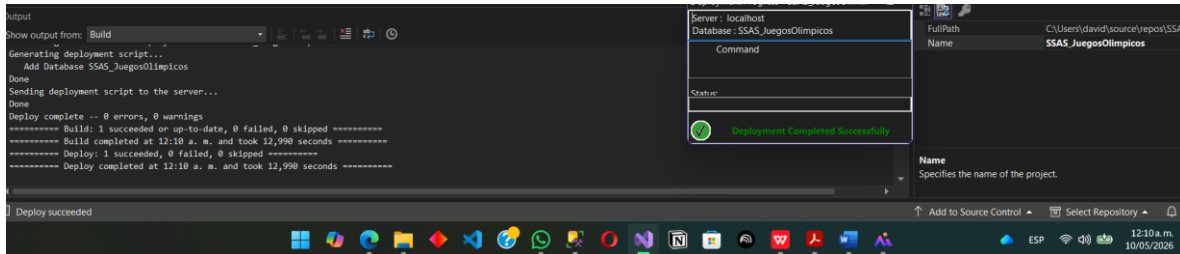
Parent member:

Expression

```
(([Measures].[Fact Participacion Count],  
[Dim Fact Participacion].[Medal].&[Bronze])  
No issues found
```



Una vez creadas todas las medidas, volvemos hacer un deploy y luego ejecutamos un proceso para la carga en el SQL



Cubo realizado, ahora hacer el Dashboard

Le indicamos cual servidor y cual Database

SQL Server database

Server

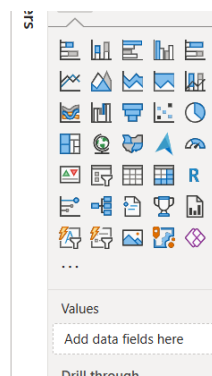
Database (optional)

Data Connectivity mode ☒ Import ☐ DirectQuery

Advanced options

OK

Cancel





Cargamos los Datos

☒ dim_atleta

☒ dim_Competicion

☒ dim_edicion

☒ dim_equipo

☒ dim_evento

☒ fact_participacion

☐ sysdiagrams

☐ fn_diagramobjects

Seleccionar tablas relacionadas

2	Judo	Judo Men's Extra-Lightweight
3	Football	Football Men's Football
4	Tug-Of-War	Tug-Of-War Men's Tug-Of-War
5	Speed Skating	Speed Skating Women's 500 metres
6	Speed Skating	"Speed Skating Women's 1,000 metres"
7	Cross Country Skiing	Cross Country Skiing Men's 10 kilometres
8	Cross Country Skiing	Cross Country Skiing Men's 50 kilometres
9	Cross Country Skiing	Cross Country Skiing Men's 10/15 kilometres
10	Cross Country Skiing	Cross Country Skiing Men's 4 x 10 kilometres
11	Cross Country Skiing	Cross Country Skiing Men's 30 kilometres
12	Athletics	Athletics Women's 100 metres
13	Athletics	Athletics Women's 4 x 100 metres Relay
14	Ice Hockey	Ice Hockey Men's Ice Hockey
15	Swimming	Swimming Men's 400 metres Freestyle
16	Badminton	Badminton Men's Singles
17	Sailing	Sailing Women's Windsurfer
18	Biathlon	Biathlon Women's 7.5 kilometres Sprint
19	Swimming	Swimming Men's 200 metres Breaststroke
20	Swimming	Swimming Men's 400 metres Breaststroke
21	Gymnastics	Gymnastics Men's Individual All-Around
22	Gymnastics	Gymnastics Men's Team All-Around
23	Gymnastics	Gymnastics Men's Floor Exercise

Cargar

Transformar datos

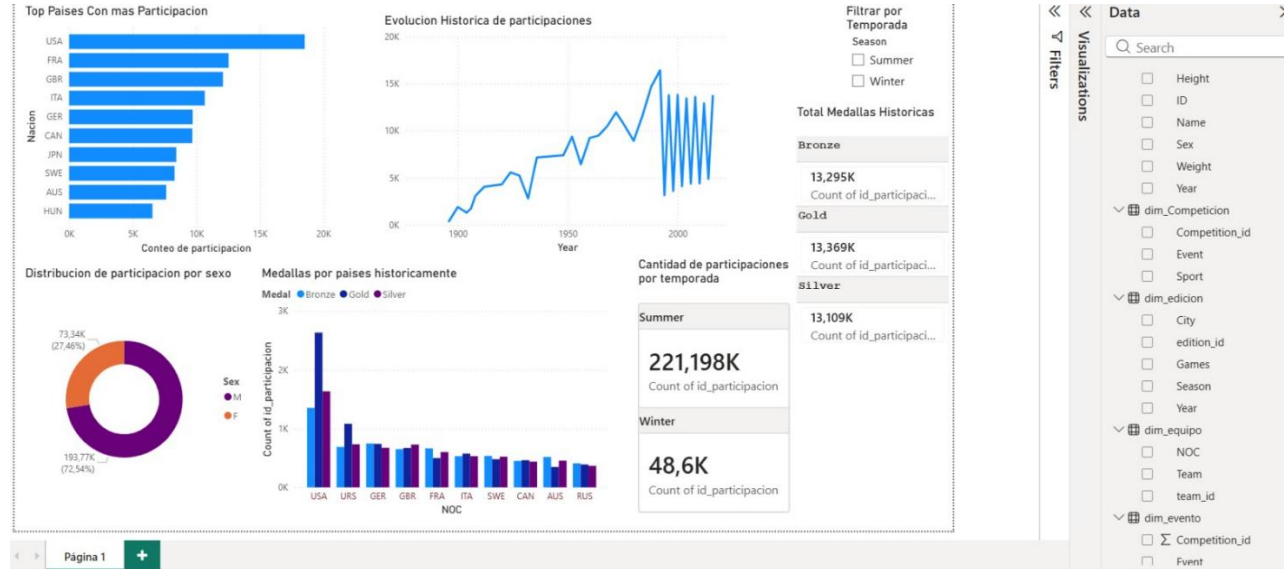
Cancelar

Haremos el Dashboard de acuerdo a estas preguntas (Seleccionadas en Rojo)

- ¿Cuál es el top de países con mayores participaciones?
- ¿Cuál es la distribución de participaciones por sexo (M/F)?
- ¿Cuál es el promedio de edad de los atletas por deporte?
- ¿Cómo ha evolucionado la participación femenina por año?
- ¿Cuáles son los 10 países con más medallas históricamente?
- ¿Cuál es el total entregado de medallas de Oro, Plata y Bronce?
- ¿En qué deportes se han entregado más medallas?
- ¿Qué ciudades han sido sede más veces de los Juegos Olímpicos?
- ¿Cómo ha evolucionado las participaciones a lo largo de la historia?



DashBoard



ETL PYTHON

```
import pandas as pd
import sqlite3
import numpy as np

print("Librerías cargadas correctamente ✅")

Librerías cargadas correctamente ✅

# Cargar archivos directamente desde Colab
athletes = pd.read_csv('Athletes_Games.csv')
edition = pd.read_csv('Edition_Games.csv')
equipos = pd.read_csv('equipos_Games.csv')
events = pd.read_csv('Competition_Games.csv')
jjoo = pd.read_csv('JJ.00.csv')

print("Athletes:", athletes.shape)
print("Edition:", edition.shape)
print("Equipos:", equipos.shape)
print("Events:", events.shape)
print("JJ00:", jjoo.shape)

Athletes: (187433, 8)
Edition: (52, 5)
Equipos: (1231, 3)
Events: (765, 3)
JJ00: (269798, 6)
```



```
print("ATHLETES ")
print(athletes.head(3))
print("\nEDITION")
print(edition.head(3))
print("\nEQUIPOS ")
print(equipos.head(3))
print("\n EVENTS")
print(events.head(3))
print("\n JJOO ")
print(jjoo.head(3))
```

ATHLETES

	athlete_id	ID	Year	Name	Sex	Age	Height	Weight
0	1	1	1992	A Dijiang	M	24.0	180.0	80.0
1	2	2	2012	A Lamusi	M	23.0	170.0	60.0
2	3	3	1920	Gunnar Nielsen Aaby	M	24.0	175.0	70.0

EDITION

	edition_id	Games	Year	Season	City
0	1	1992 Summer	1992	Summer	Barcelona
1	2	2012 Summer	2012	Summer	London
2	3	1920 Summer	1920	Summer	Antwerpen

EQUIPOS

	team_id	Team	NOC
0	1	China	CHN
1	2	Denmark	DEN
2	3	Denmark/Sweden	DEN



=== NULOS POR TABLA ===

Athletes:

athlete_id	0
ID	0
Year	0
Name	0
Sex	0
Age	0
Height	0
Weight	0

dtype: int64

Edition:

edition_id	0
Games	0
Year	0
Season	0
City	0

dtype: int64

Equipos:

team_id	0
Team	0
NOC	0

dtype: int64

Events:

Competition_id	0
Sport	0
Event	0

dtype: int64

JJ00:

id_participacion	0
Medal	0
athlete_id	0
team_id	0
edition_id	0
Competition_id	0

dtype: int64



```
# Limpiar athletes - convertir tipos y manejar nulos
athletes['Age'] = pd.to_numeric(athletes['Age'], errors='coerce')
athletes['Height'] = pd.to_numeric(athletes['Height'], errors='coerce')
athletes['Weight'] = pd.to_numeric(athletes['Weight'], errors='coerce')

# Limpiar JJOO - estandarizar medallas
jjoo['Medal'] = jjoo['Medal'].fillna('No Medal')

print("Nulos en Age:", athletes['Age'].isnull().sum())
print("Nulos en Height:", athletes['Height'].isnull().sum())
print("Nulos en Weight:", athletes['Weight'].isnull().sum())
print("Limpieza completada ✅")

Nulos en Age: 0
Nulos en Height: 0
Nulos en Weight: 0
Limpieza completada ✅
```

```
# Indicador 1 - Total participaciones
total_participaciones = len(jjoo)

# Indicador 2 - Total medallas
total_medallas = len(jjoo[jjoo['Medal'] != 'No Medal'])

# Indicador 3 - Porcentaje de medallistas
pct_medallistas = (total_medallas / total_participaciones) * 100

# Indicador 4 - Promedio edad, altura, peso
avg_edad = athletes['Age'].mean()
avg_altura = athletes['Height'].mean()
avg_peso = athletes['Weight'].mean()

print(f"Total participaciones: {total_participaciones:,}")
print(f"Total medallas: {total_medallas:,}")
print(f"% medallistas: {pct_medallistas:.2f}%")
print(f"Promedio edad: {avg_edad:.1f} años")
print(f"Promedio altura: {avg_altura:.1f} cm")
print(f"Promedio peso: {avg_peso:.1f} kg")

Total participaciones: 269,798
Total medallas: 39,773
% medallistas: 14.74%
Promedio edad: 25.8 años
Promedio altura: 176.2 cm
Promedio peso: 72.0 kg
```




```
# Crear base de datos SQLite
conn = sqlite3.connect('JuegosOlimpicos.db')

# Guardar cada tabla
athletes.to_sql('dim_atleta',          conn, if_exists='replace', index=False)
edition.to_sql('dim_edicion',          conn, if_exists='replace', index=False)
equipos.to_sql('dim_equipo',           conn, if_exists='replace', index=False)
events.to_sql('dim_evento',            conn, if_exists='replace', index=False)
jjoo.to_sql('fact_participacion',      conn, if_exists='replace', index=False)

print("Base de datos SQLite creada ✅")

# Verificar
for tabla in ['dim_atleta', 'dim_edicion', 'dim_equipo', 'dim_evento', 'fact_participacion']:
    count = pd.read_sql(f"SELECT COUNT(*) as filas FROM {tabla}", conn)
    print(f"{tabla}: {count['filas'][0]:,} filas")

conn.close()
```

Base de datos SQLite creada ✅
dim_atleta: 187,433 filas
dim_edicion: 52 filas
dim_equipo: 1,231 filas
dim_evento: 765 filas
fact_participacion: 269,798 filas



DASHBOARD PYTHON

```
import plotly.express as px
import plotly.graph_objects as go

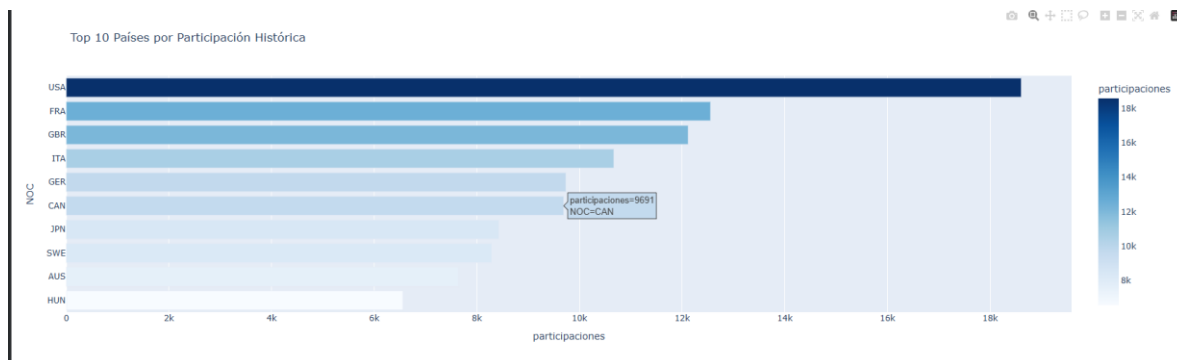
# Reconectar a SQLite
conn = sqlite3.connect('JuegosOlimpicos.db')

print("Plotly listo ✅")
```

Plotly listo ✅

```
# Consulta
df_paises = pd.read_sql("""
    SELECT e.NOC, COUNT(*) as participaciones
    FROM fact_participacion f
    JOIN dim_equipo e ON f.team_id = e.team_id
    GROUP BY e.NOC
    ORDER BY participaciones DESC
    LIMIT 10
""", conn)

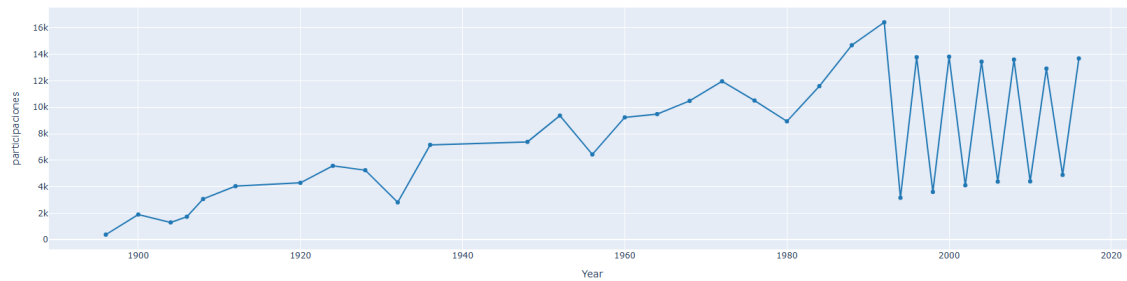
# Gráfico
fig1 = px.bar(df_paises,
              x='participaciones',
              y='NOC',
              orientation='h',
              title='Top 10 Países por Participación Histórica',
              color='participaciones',
              color_continuous_scale='Blues')
fig1.update_layout(yaxis={'categoryorder':'total ascending'})
fig1.show()
```





```
fig2 = px.line(df_años,  
              x='Year',  
              y='participaciones',  
              title='Evolución Histórica de Participaciones 1896-2016',  
              markers=True)  
fig2.update_traces(line_color='#1f77b4')  
fig2.show()
```

Evolución Histórica de Participaciones 1896-2016

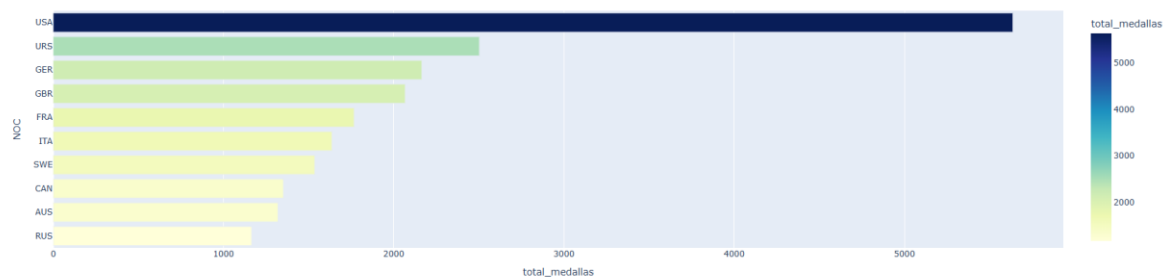


```
#### a-sex  
###, conn)  
  
fig3 = px.pie(df_sexo,  
              values='cantidad',  
              names='Sex',  
              title='Distribución de Participación por Género',  
              hole=0.4,  
              color_discrete_map={'M': '#1f77b4', 'F': '#ff7f0e'})  
fig3.show()
```

Distribución de Participación por Género

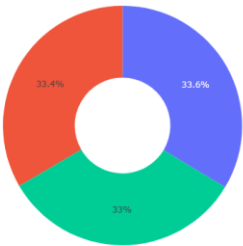


Top 10 Países con más Medallas (Oro, Plata, Bronce) Históricamente





Distribución de Medallas (Oro, Plata, Bronce)



■ Gold
■ Bronze
■ Silver

PUNTO 6.

1. Herramientas utilizadas en el proyecto

El proyecto implementó una arquitectura BI completa con dos enfoques paralelos: uno basado en el stack Microsoft (SSIS + SSAS + Power BI) y otro con herramientas de código abierto (Python + SQLite + Plotly). A continuación se describen las cuatro herramientas principales.

HERRAMIENTAS ETL

SSIS ETL — Stack Microsoft		Python ETL ETL — Código abierto	
Herramienta	SQL Server Integration Services	Herramienta	pandas + sqlite3 + numpy
Proyecto	ETL_JuegosOlimpicos.dtsx	Proyecto	ETL_Python_Big_Data.ipynb
Destino	SQL Server (JuegosOlimpicos DB)	Destino	SQLite (JuegosOlimpicos.db)
Entorno	Visual Studio + SSDT	Entorno	Google Colab / Jupyter
Despliegue	.ispac → SSIS Catalog	Despliegue	Notebook ejecutable



HERRAMIENTAS DE VISUALIZACIÓN

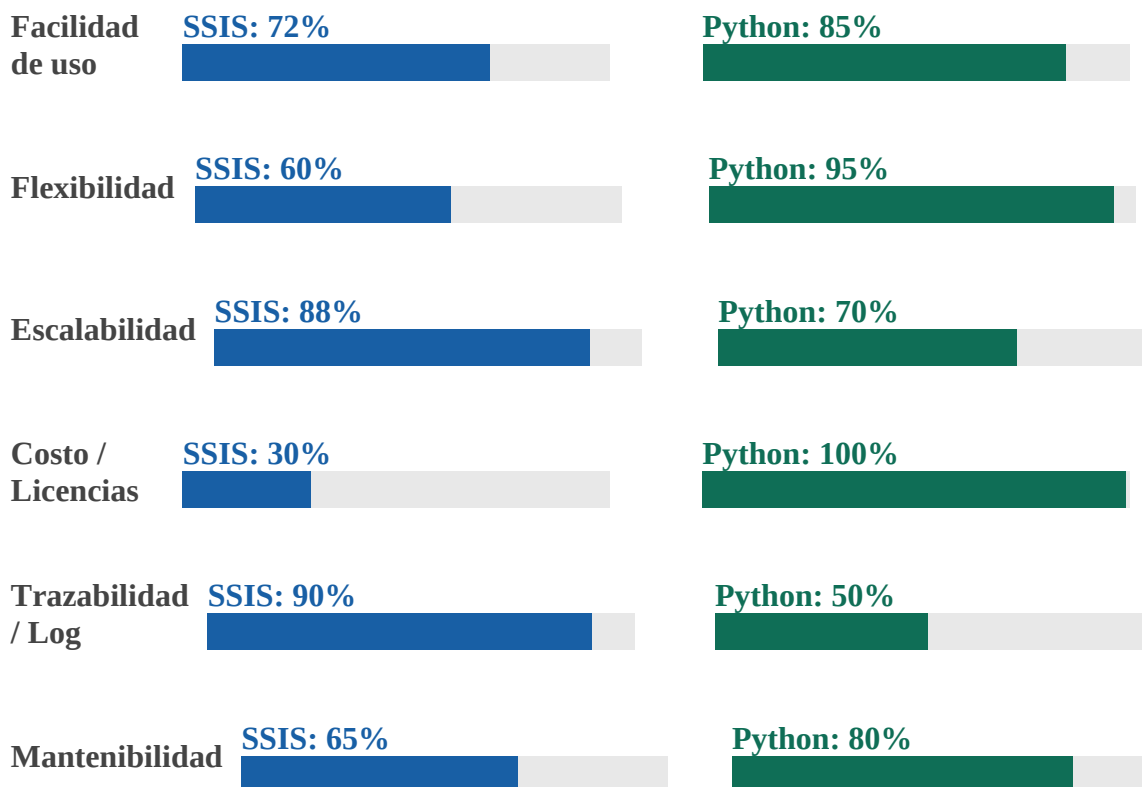
Power BI + SSAS Visualización — Stack Microsoft	
Herramienta	Power BI Desktop + SSAS
Cubo	Juegos Olimpicos.cube
Dimensiones	Atleta, Edición, Equipo, Competición
Fact table	Fact Participacion
Fuente	Conexión directa a SSAS

Python (Plotly) Visualización — Código abierto	
Herramienta	Plotly Express + SQLite
Gráficos	Bar, Line, Pie, Grouped Bar
Datos	SQL directo sobre SQLite
Entorno	Google Colab / Jupyter
Exportación	HTML interactivo / PNG



2. Comparación ETL: SSIS vs Python

2.1 Evaluación por criterio



2.2 Análisis detallado

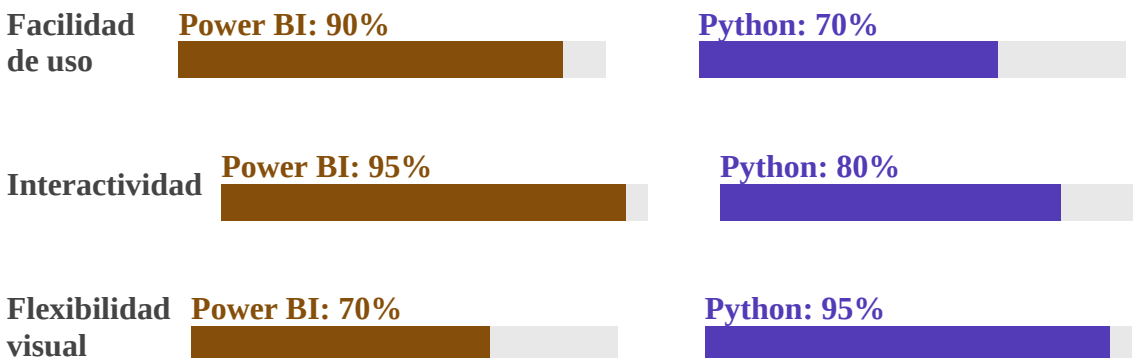
Criterio	SSIS	Python (pandas + sqlite3)
Facilidad	Interfaz visual drag-and-drop. Requiere Visual Studio + SSDT + SQL Server. Alta curva de configuración inicial pero flujos intuitivos una vez configurado.	Sin instalación (Google Colab). Basta ejecutar celdas. Curva baja para perfiles técnicos. No requiere licencias.
Flexibilidad	Limitado a transformaciones predefinidas (Sort, Merge, Derived Column). Lógica compleja necesita	Ilimitada: pandas permite cualquier transformación. Limpieza Age/Height/Weight con

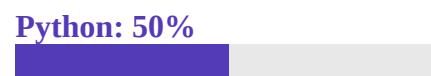


Criterio	SSIS	Python (pandas + sqlite3)
	componentes Script o SQL adicional.	pd.to_numeric(errors='coerce') en una sola línea.
Escalabilidad	Diseñado para grandes volúmenes empresariales. Soporta paralelismo, particionado y múltiples fuentes heterogéneas simultáneas.	SQLite válido para datos pequeños-medianos. Para Big Data real se requiere Spark o Dask. El notebook carga todos los CSV en memoria RAM.
Aplicabilidad	Entornos corporativos Microsoft: SQL Server, Azure Data Factory. Ideal para flujos recurrentes en producción con orquestación.	Proyectos académicos, exploración, pipelines ad-hoc, Machine Learning. Compatible con cualquier SO sin licencias ni infraestructura.
Monitoreo	Logs nativos en el SSIS Catalog: historial de ejecuciones, alertas por correo y reintentos automáticos configurables.	Solo impresión por consola con print(). Sin logs persistentes salvo implementación manual de logging o MLflow.
Control versiones	Archivos .dtsx en XML gestionados con Git, pero los diffs son difíciles de leer al ser XML generado automáticamente.	Notebooks .ipynb versionados con Git de forma natural. Diffs legibles celda a celda; compatible con nbdime.

3. Comparación Visualización: Power BI vs Python

3.1 Evaluación por criterio





3.2 Análisis detallado

Criterio	Power BI + SSAS	Python (Plotly Express)
Facilidad	Sin código: arrastrar campos del cubo SSAS al canvas genera el visual. Conexión directa con las 4 dimensiones y la fact table.	Requiere consultas SQL sobre SQLite y código Plotly por cada gráfico. Cada visual es una celda independiente en el notebook.
Interactividad	Cross-filtering nativo: clic en un país filtra todos los demás visuales automáticamente. Slicers por Year, Season y City sin código.	Plotly ofrece zoom, hover y leyenda clicable. Sin cross-filtering entre gráficos salvo desarrollo adicional con Dash.
Integración SSAS	Conexión directa al cubo analítico con jerarquías precalculadas. Aprovecha medidas MDX y agrupaciones del cubo Juegos Olimpicos.cube.	Sin integración con cubos OLAP. Usa SQL directo sobre SQLite sin jerarquías ni medidas analíticas precalculadas.
Escalabilidad	El cubo SSAS pre-agrega los datos. Millones de filas en fact_participacion se consultan en milisegundos desde Power BI.	SQLite válido para el dataset del proyecto. No escala a producción: cargar todo en pandas agota memoria con datasets grandes.
Aplicabilidad	Dashboards empresariales: compartir vía Power BI Service, refrescos automáticos, control de acceso por roles de negocio.	Análisis exploratorio y científico: combinar visualización con ML/estadística en el mismo



Criterio	Power BI + SSAS	Python (Plotly Express)
		notebook. Exportable como HTML standalone.
Personalización	Limitado a visuales del marketplace. Personalización avanzada requiere DAX o incrustar R/Python dentro de Power BI.	Total: colores, fuentes, anotaciones, animaciones. <code>color_discrete_map</code> define exactamente Oro=#FFD700, Plata=#C0C0C0, Bronce=#CD7F32.

4. Conclusiones y recomendaciones

- Con esto podemos concluir que con Python se demostró una mayor agilidad para este proyecto, configuración rápida y sencilla en Google Colab de forma gratuita y con limpieza de datos expresiva perfecto para ejercicios académicos. SSIS es perfecto para trabajar en entornos corporativos con flujos mas recurrentes, con un Logging mas robusto. Cual es mejor depende del contexto que se requiera, para temas de aprendizaje y exploración Python y para temas Corporativos y empresariales SSIS.
- Power BI + SSAS ofrece una experiencia mas rica para BI: viene con Crossfiltering nativo y una integración con cubo analítico. Python con Plotly es excelente para temas de exploración mas técnica con posibilidad de una personalización avanzada, pero a su vez requiere de mas código para poder lograr una interactividad.
- En proyectos reales ambos enfoques son complementarios. Python sirve mas para aprendizaje y exploración, mientras SSIS para productividad con el Pipeline SSAS es perfecto para motor analítico y Power BI para el Dashboard y la reporteria ejecutiva. Este proyecto implemento exactamente esta arquitectura end to end, representando así una solución enfocada en BI profesional.



4.1 Cuándo usar cada herramienta

Escenario	Herramienta recomendada	Razón
Prototipado rápido / académico	Python (pandas + Plotly)	Sin licencias, configuración inmediata, flexibilidad total
Pipeline ETL en producción corporativa	SSIS	Logging, reintentos, monitoreo y soporte empresarial
Dashboard para usuarios de negocio	Power BI	Interactividad nativa, distribución vía Power BI Service
Análisis estadístico / Machine Learning	Python (pandas + Plotly/Dash)	Integración con scikit-learn, statsmodels, TensorFlow
Grandes volúmenes (>100M filas)	SSIS + SSAS + Power BI	Pre-agregación en cubo, consultas MDX en milisegundos
Proyecto con equipo mixto (TI + negocio)	Stack completo Microsoft	Gobierno de datos, seguridad por roles y colaboración



12. CONCLUSIONES

Sobre el ETL y la preparación de datos:

El proceso de extracción, transformación y carga demostró que la calidad de los datos es el factor más crítico en cualquier proyecto de BI. Los registros históricos olímpicos presentaron inconsistencias significativas — valores nulos en Age (22%), Height (22%) y Weight (23%) — que reflejan las limitaciones de los registros del siglo XIX y exigieron decisiones metodológicas fundamentadas, como la imputación por media, para preservar la integridad del análisis sin perder representatividad.

Sobre el modelo analítico: El modelo estrella implementado en SQL Server y SSAS demostró ser la arquitectura óptima para el análisis multidimensional de datos olímpicos. La separación entre tabla de hechos (269,798 participaciones) y dimensiones (atletas, ediciones, equipos, competiciones) permitió consultas eficientes y la construcción de medidas calculadas relevantes como el porcentaje de medallistas (aproximadamente 14.8% del total de participantes).

Sobre el análisis descriptivo:

Los resultados evidencian una marcada evolución del olimpismo: las participaciones crecieron de 241 atletas en 1896 a más de 13,000 en 2016. La brecha de género se ha reducido significativamente — de una participación femenina casi nula en los primeros Juegos a representar el 27.5% del total histórico — aunque persiste una desigualdad que el movimiento olímpico continúa abordando. Estados Unidos, Francia y Gran Bretaña lideran históricamente en participaciones, mientras que en medallas destacan potencias como la URSS y Alemania del Este en períodos específicos.

Sobre la comparación tecnológica:

SSIS y Power BI ofrecen ventajas significativas en entornos empresariales con infraestructura Microsoft, especialmente por su interfaz visual que reduce la curva de aprendizaje. Sin embargo, Python demostró mayor flexibilidad y reproducibilidad — el notebook en Google Colab permite replicar todo el proceso ETL y las visualizaciones sin licencias adicionales, lo que lo hace más apropiado para equipos técnicos y proyectos de ciencia de datos. Ambos ecosistemas son complementarios y la elección debe basarse en el contexto organizacional.



Sobre el modelo predictivo:

Los datos físicos y demográficos por sí solos presentan una correlación baja con la obtención de medallas, lo que confirma que el éxito olímpico es un fenómeno multifactorial que incluye variables no capturadas en este dataset, como el nivel de entrenamiento, la inversión del país, la competencia en cada disciplina y factores psicológicos. Esto refuerza la necesidad de modelos predictivos especializados por disciplina deportiva

13. REFERENCIAS.

Griffin, R. (2018). *120 years of Olympic history: athletes and results* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/heesoo37/120-years-of-olympic-history-athletes-and-results>

Microsoft Corporation. (2024). *Power BI documentation*. Microsoft Docs. <https://docs.microsoft.com/power-bi>

Microsoft Corporation. (2024). *SQL Server Integration Services documentation*. Microsoft Docs. <https://docs.microsoft.com/sql/integration-services>

Microsoft Corporation. (2024). *Analysis Services documentation*. Microsoft Docs. <https://docs.microsoft.com/analysis-services>

Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library*. <https://plotly.com/python>